

Multi-Layer Metamodeling for Verifiable Credential Ecosystem Design: Formalizing Cross-Layer Constraints with Graph Predicates

Anonymous Author(s)

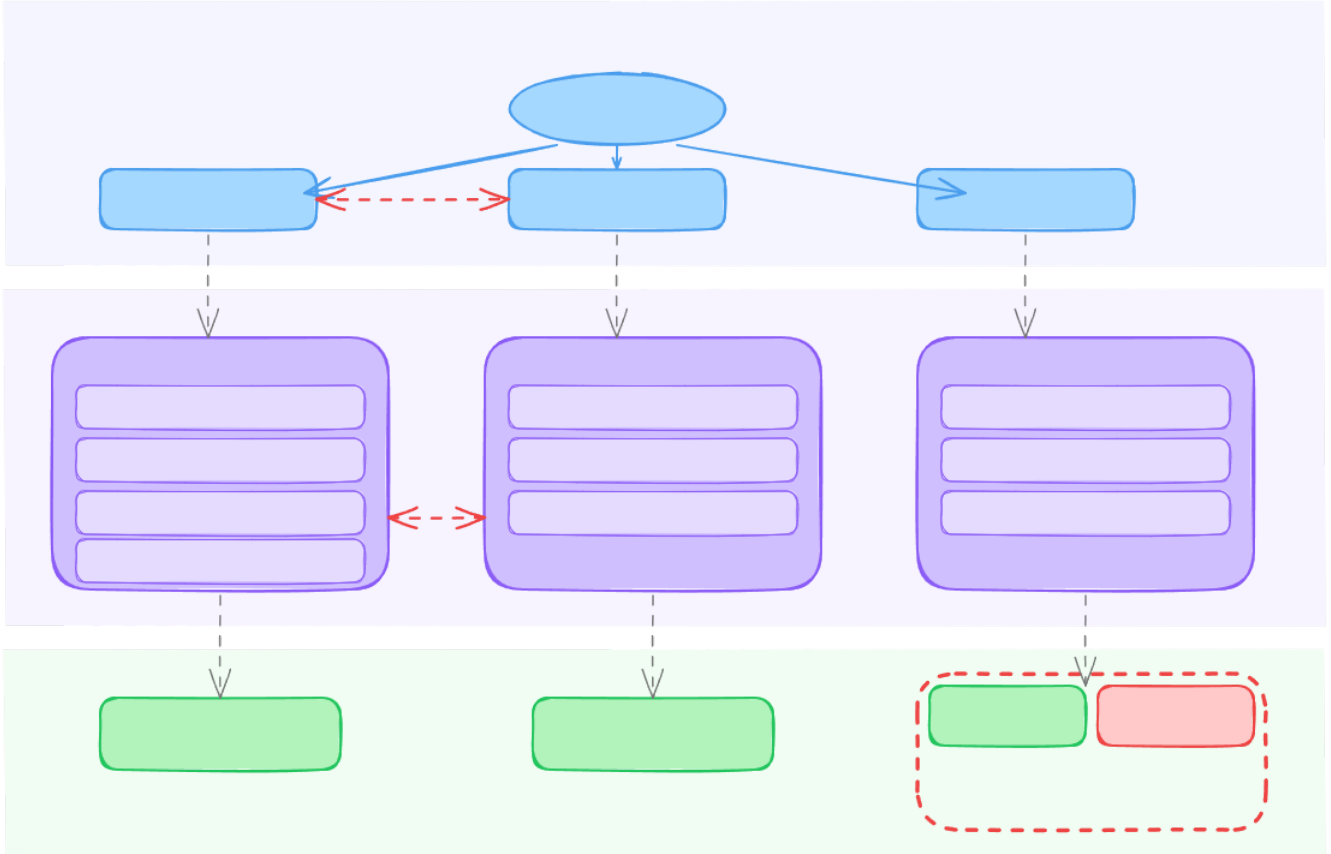


Figure 1: A housing subsidy scenario across three metamodel layers. Domain facts at the domain concept layer are grouped into credentials with aligned subjects at the credential schema layer. At the format-specific layer, eIDAS format mandates and GDPR data minimization impose contradictory requirements on the income credential; cross-credential predicates lack privacy-preserving enforcement entirely.

Abstract

Emerging digital credential ecosystems will allow persons and organizations to receive, hold, and selectively present cryptographically verifiable claims about themselves. Such ecosystems are being deployed under diverse and sometimes contradictory governance frameworks - from government-regulated EU Digital Identity Wallets to community-governed decentralized identity ecosystems based on self-sovereign identity principles. Designing these ecosystems requires satisfying constraints that span multiple abstraction layers - from domain-level claim properties through credential schema structuring to format-specific privacy capabilities - yet these constraints are currently scattered across sources such as

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MODELS 2026, Malaga, Spain

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/10
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

W3C standards, EU implementing regulations, or community design guidelines, with no formal means to check their consistency or detect design errors.

We present a layered metamodel and formalized constraint set for credential ecosystem design, spanning domain-level claim properties, abstract credential schemas, and format-specific credential representations. The metamodel is grounded in the W3C Verifiable Credentials Data Model 2.0; cross-layer constraints are formalized as graph predicates in the Refinery partial graph modeling framework. We validate metamodel coverage against the W3C specification, constraint expressiveness against requirements from W3C and EU regulatory sources, and error detection against known credential design anti-patterns. Our results show that constraints originating from different governance frameworks can be formally conflicting, and that design errors spanning multiple layers - undetectable by single-layer inspection — become visible through the integrated formalization.

CCS Concepts

- **Software and its engineering** → **Context specific languages;**
- **Security and privacy** → **Pseudonymity, anonymity and untraceability; Privacy-preserving protocols.**

Keywords

metamodeling, verifiable credentials, design space exploration, partial models, graph constraints

ACM Reference Format:

Anonymous Author(s). 2026. Multi-Layer Metamodeling for Verifiable Credential Ecosystem Design: Formalizing Cross-Layer Constraints with Graph Predicates. In *Proceedings of ACM/IEEE 29th International Conference on Model Driven Engineering Languages and Systems (MODELS 2026)*. ACM, New York, NY, USA, ?? pages. <https://doi.org/10.1145/nmnnnnn.nnnnnnn>

1 Introduction

Emerging digital credential ecosystems allow persons and organizations to receive, hold, and selectively present cryptographically verifiable claims [?]. Such ecosystems are deployed under diverse governance frameworks — from government-regulated EU Digital Identity Wallets [?] to community-governed decentralized identity systems [?]. Designing these ecosystems requires satisfying constraints that span multiple abstraction layers — from domain-level claim properties through credential schema structure to format-specific privacy capabilities — yet no multi-layer formalization captures the cross-layer constraints spanning domain semantics, credential structure, and format-specific representation.

Constraints from W3C standards, EU implementing regulations, and community design guidelines interact across abstraction layers, and their combined effect on a single credential ecosystem design is not predictable from any individual governance source [?]. For example, the eIDAS Architecture Reference Framework mandates a specific credential format for government attestations; GDPR requires that privacy-sensitive claims disclose only the minimum necessary information; neither regulation anticipates the other, yet their joint effect on a single income credential may be unsatisfiable. Unlike hierarchical requirement systems in safety-critical domains,

these governance sources are independent peers whose design goals may be formally irreconcilable.

Design errors that span multiple layers go unnoticed because no multi-layer formalization captures the cross-layer constraints needed to make them expressible. Existing tools operate at a single layer — JSON Schema validators check credential structure, format-specific conformance tools verify encoding constraints, governance frameworks define requirements in isolation — but none checks cross-layer consistency. A credential schema may be well-formed when inspected in isolation, yet violate a cross-layer constraint that links domain-level claim semantics to format-specific privacy capabilities.

We make the following contributions:

1. A **three-layer metamodel** for credential ecosystem design, grounded in the W3C Verifiable Credentials Data Model 2.0, spanning claim properties, credential schemas, and format-specific representations (??).
2. A **formalization of cross-layer constraints** as graph predicates in the Refinery partial graph modeling framework (??).
3. A **three-axis validation** demonstrating metamodel coverage against the W3C specification, constraint expressiveness against W3C and EU regulatory sources, and error detection against known credential design anti-patterns, including the detection of two cross-layer design errors — a governance conflict and a format expressiveness gap — undetectable by single-layer analysis (??).

Cross-layer constraints are formalized as graph predicates in Refinery [?], which generates diverse model instances satisfying these constraints — or reveals that no consistent design exists when constraints from different governance frameworks are formally contradictory. The framework supports three usage modes: *consistency checking* confirms that a partial or complete design satisfies all constraints, *error identification* names specific constraint violations, and *design space exploration* generates diverse valid configurations or proves that none exist. A housing subsidy credential scenario serves as the running example throughout.

%% Structure paragraph — regenerate with skills/structure_paragraph when sections change. %%

?? introduces the W3C Verifiable Credentials Data Model, multi-layer modeling concepts, and the Refinery framework. ?? motivates the multi-layer design problem through the running example, defines the three usage modes, and walks through a designer's workflow. ?? presents the three-layer metamodel and formalizes cross-layer constraints as Refinery graph predicates. ?? validates metamodel coverage, constraint expressiveness, and error detection through qualitative elaboration and quantitative scalability measurement. ?? positions the contribution against related work, and ?? discusses limitations and future directions.

2 Background

2.1 W3C Verifiable Credentials Data Model 2.0

The W3C Verifiable Credentials Data Model (VCDM) 2.0 [?] defines a verifiable credential as a tamper-evident set of claims made by an issuer about one or more credential subjects. A *claim* is a subject-property-value triple asserting a characteristic of the credential

subject — for example, that an applicant’s monthly income exceeds a threshold. The *issuer* is the entity asserting the claims; the *holder* possesses the credential and presents it to a *verifier*. Each credential carries a digital signature or comparable cryptographic evidence that binds the claims to the issuer and allows a verifier to confirm their integrity. VCDM 2.0 standardizes this structural vocabulary: the abstract data model for credential issuance, holding, and verification. It does not, however, prescribe the domain-level semantics of claim values, the encoding format in which credentials are serialized, or the privacy capabilities available during presentation. This gap — between structural vocabulary and the domain-specific, format-specific concerns that govern real deployments — motivates the multi-layer separation developed in ??.

VCDM credentials can be secured and serialized in multiple formats with distinct cryptographic foundations and privacy capabilities. SD-JWT-VC [?] provides hash-based *selective disclosure*, allowing a holder to reveal a chosen subset of claims while withholding others. AnonCreds [?] uses Camenisch–Lysyanskaya signatures to support both selective disclosure and *predicate proofs* — zero-knowledge proofs that a claim value satisfies a given predicate (e.g., $\text{monthly_income} \geq \text{threshold}$) without disclosing the value itself. JSON-LD credentials with Data Integrity Proofs [?] support selective disclosure via the BBS cryptosuite [?] but, as of the current W3C Data Integrity specification, lack predicate proof capability (the BBS cryptosuite specification does not currently define predicate proof operations, though the underlying BBS+ signature scheme admits extensions that could support them). The ISO 18013-5 mobile document format (mdoc) [?] is mandated alongside SD-JWT-VC by the eIDAS 2.0 Architecture Reference Framework for EU wallet attestations [?]; since both formats lack predicate proof support, subsequent sections use SD-JWT-VC as the representative format for the governance conflict analysis. The two formats differ on other capability predicates (notably offline verification); the substitution is without loss of generality only with respect to predicate proof capability.

2.2 Multi-Layer Modeling

When a modeling problem spans multiple abstraction layers, each layer defines its own types, instance(s), and well-formedness rules. Multi-level metamodeling [?] organizes such hierarchies through successive instantiation controlled by potency annotations. The three layers in ?? — domain concepts, credential schemas, and format-specific representations — are abstraction layers in this sense, but they are not related by instantiation. Each layer is an independently governed concern space whose constraints originate from distinct normative sources (W3C standards, EU regulations, format specifications) not designed for joint satisfaction. Cross-layer relationships are formalized as typed trace references and graph predicates over partial models (??), not as potency annotations.

Layer definitions in ?? use Ecore-style class diagrams with typed references and containment hierarchies as the metamodeling notation. Layer instance(s) are *partial models*: model elements may have definite (*must*), absent (*must not*), or open (*unknown*) status for any reference or class membership, enabling reasoning over specifications where not all design decisions have been made. Cross-layer

trace relationships are formalized as typed references between partial models at adjacent layers, capturing the correspondence between a domain concept and the credential-layer claims that represent it.

2.3 Partial Graph Modeling with Refinery

Credential ecosystem designs are inherently incomplete: schema elements, format assignments, and governance constraints are introduced incrementally as standards evolve and regulatory frameworks are adopted. Bounded model finding — Alloy [?] for relational logic, OCL for UML class models — requires fully specified instance(s) and checks constraints after generation; partial graph modeling evaluates constraints *during* refinement of incomplete specifications, pruning inconsistent design choices before they propagate. This distinction is critical when governance constraints from independent sources must be checked jointly over a design that is not yet fully determined.

Refinery is a partial graph modeling framework in which design specifications are expressed as partial models and graph predicates serve as a first-class constraint language [?]. A partial model assigns a four-valued interpretation — *must* (committed true), *must not* (committed false), *may* (possibly true), and *may not* (possibly false) — to every node, edge, attribute value, and class membership, enabling the framework to reason over incomplete specifications where both structural and data-level decisions remain open. Graph predicates define structural constraints and derived properties over these partial models; the framework evaluates them over partial interpretations; a constraint violation is confirmed only when the error predicate evaluates to *must* — that is, it holds under every possible refinement of the partial model. Given a partial specification, Refinery’s design space exploration generates diverse concrete models satisfying all constraints, or determines that no consistent completion exists [?]. This generation is refinement-based: every generated model is guaranteed to satisfy all specified constraints, so violations are excluded during generation rather than detected post-hoc.

The cross-layer constraint formalization in ?? uses four Refinery mechanisms. An *error predicate* defines a structural pattern that must not hold in any well-formed model; when an error predicate is satisfied, it identifies a specific design anti-pattern — for example, a credential containing no claims. A *propagation rule* derives new facts from the current partial interpretation to guide model generation; a special case is *negative elimination*, where the rule removes design choices that would necessarily violate a constraint. A *shadow predicate* records derived information — such as entity alignment between two credential subjects — without constraining the model, making implicit relationships available for inspection. Finally, *scope constraints* bound the number of instances of each type, controlling the size and shape of generated models. Together, these mechanisms operate over partial interpretations, systematically resolving unknown elements into definite values while maintaining all constraints.

3 Overview

3.1 Motivation

Consider a government housing subsidy where eligibility requires credentials from three independent authorities: family status from a civil registry, property records from a land registry, and income from an employer.¹ The required property size depends on the number of children [?] (the decree prescribes minimum floor areas of 40, 50, 60, 70, and 80 m² for one through five or more children, respectively) — a constraint that spans two credentials — and the income check must satisfy both EU format mandates and data protection requirements.

At the domain concept layer, the applicant’s facts — number of children, property floor area, monthly income — form an information graph with domain-level constraints: the minimum floor area is a function of the number of children [?], and income must exceed a regulatory threshold. At the credential schema layer, these facts are distributed across three credentials issued by independent authorities, each with its own credential subject. All three subjects must be aligned — they refer to the same applicant — and each claim must trace to the corresponding domain-level property. At the format-specific layer, EU regulations mandate specific credential formats for government-issued attestations [?], while data protection law requires that privacy-sensitive checks — such as whether income exceeds a threshold — disclose only the minimum necessary information [?] [?].

Inspected in isolation, each layer is well-formed: the domain properties satisfy their value constraints, the credential schemas conform to VCDM structural rules, and each format meets its own specification. Cross-layer analysis, however, reveals two distinct problems. First, the income credential cannot simultaneously satisfy the eIDAS ARF format mandate (requiring SD-JWT-VC [?]) and GDPR data minimization (requiring predicate proof capability [?]): SD-JWT-VC supports hash-based selective disclosure but not predicate proofs [?]. Second, the floor area constraint — minimum area as a function of number of children — requires cross-credential predicate evaluation spanning the family status and property credentials, which no deployed format supports in zero-knowledge. Neither problem is visible when any single layer is inspected alone.

The constraints in this scenario originate from governance frameworks that were not designed to be jointly satisfied: W3C VCDM [?] defines structural conformance rules for credentials, the eIDAS ARF [?] mandates specific credential formats for government attestations, GDPR [?] requires data minimization, and format specifications [?] [?] define what each format can and cannot express. Unlike hierarchical requirement systems in safety-critical domains, where stakeholders cooperate within a defined authority structure, these governance sources are independent and their design goals may be formally irreconcilable. To the best of our knowledge, no existing tool or methodology checks whether their constraints can be simultaneously satisfied for a given credential ecosystem design.

¹Based on the Hungarian Family Housing Subsidy (Családi Otthonteremtési Kedvezmény, CSOK), simplified. Additional credentials required in practice (tax clearance, criminal record check) are omitted; these involve simple status checks that do not introduce cross-credential arithmetic or predicate proof requirements, and their inclusion would not affect the headline results.

Functional overview: partial design + constraints → metamodel
+ graph predicates → OK / NOT_OK(errors) /
GENERATED(design) / UNVAILABLE(reasons)

Figure 2: Functional overview of the framework. A partial credential ecosystem design and constraint set are evaluated against the three-layer metamodel via graph predicates, producing one of three outcomes: consistency confirmed, specific violations identified, or valid configurations generated (or proven nonexistent).

Credential ecosystem design requires a formal framework that captures constraints at multiple abstraction layers and from multiple governance sources, enabling designers to determine whether a given configuration is jointly satisfiable. Recent surveys of the credential ecosystem landscape [?] identify no existing approach that provides this capability — existing formalizations target individual layers or individual governance sources, not their cross-layer interaction.

3.2 Functional Overview

The approach takes as input a *partial credential ecosystem design* — entities, credentials, claim mappings, subject bindings, and tentative format assignments — together with the constraint set derived from the metamodel and applicable governance frameworks. The metamodel organizes these elements across three layers (domain concepts, credential schemas, format-specific representations), and cross-layer constraints are formalized as graph predicates that the Refinery partial graph modeling framework [?] can evaluate or enforce.

The framework supports three usage modes. In *consistency checking*, the designer submits a complete or partial design and receives confirmation that all constraints are satisfied (**OK**). In *error identification*, the framework evaluates error predicates against the design and returns specific constraint violations (**NOT_OK(errors)**) — for example, identifying that the income credential’s format assignment violates a governance constraint. In *design space exploration*, the designer provides a partial specification with open design choices (such as unassigned credential formats) and the framework either generates diverse valid configurations (**GENERATED(possible_design)**) or proves that no configuration satisfying all constraints exists (**UNVAILABLE(reasons)**).

3.3 Usage Workflow

A designer specifies the housing subsidy credential ecosystem: three credentials with their claim mappings, subject bindings, and tentative format assignments. Running error identification reveals a constraint violation on the income credential — the eIDAS ARF

format mandate and GDPR data minimization requirement conflict under SD-JWT-VC, which lacks predicate proof capability. The designer adjusts the design — restructuring the income claim as a pre-computed boolean (income above threshold) — and re-checks. Finding the modified design consistent, the designer runs design space exploration with format assignments left open, generating alternative valid configurations. The framework produces two configurations that satisfy all governance constraints, or returns UN-VIABLE if the full constraint set — including the original predicate proof requirement — admits no solution.

4 Approach

Credential ecosystem design involves three distinct concerns: what domain-level facts exist and how they relate, how those facts are grouped into credentials with defined subjects and claims, and which concrete credential format each credential uses. The meta-model separates these concerns into three layers — the *domain concept layer* (DCL), the *credential schema layer* (CSL), and the *format-specific layer* (FSL). The layered structure follows the multi-layer modeling principles introduced in ?? : each layer is derived from the one above and carries its own intra-layer constraints. Constraints within and across layers are formalized as graph predicates that the Refinery framework (??) can evaluate, yielding the three usage modes defined in ??. ?? shows the complete metamodel; the definitions below correspond to its elements layer by layer.

4.1 Domain Concept Layer

The domain concept layer models domain-level facts as a typed information graph. The abstract metaclass Entity has two concrete subclasses: Subject and Value. A Subject is an entity that can anchor a credential — it represents the person, organization, or thing about which claims are made. A Value is an entity that serves as the target of a property. Each Entity contains zero or more Prop instances; each Prop holds exactly one Value through a containment reference and carries a trace link to the credential schema layer (??). The ternary predicate statement(s, p, v) holds when subject s owns property p and p contains value v , with the well-formedness condition $s \neq v$. The distinction between Subject and Value is inferred structurally: a propagation rule classifies any Entity with no incoming value reference as a Subject — it is a root of the property tree, and therefore a candidate credential subject at the CSL layer. The complete Refinery encoding is provided in the supplementary material.

A credential ecosystem must cover all domain-level facts — an entity unreachable from the rest of the graph represents a fact that no credential path connects to the subject, and therefore cannot be verified. The non_connected error predicate enforces this: it flags any pair of entities not transitively reachable through the neighbours relation, where two entities are neighbours if a statement connects them in either direction. The no_self_loop propagation rule prevents an entity from serving as both the owner and the value of the same property — a structurally meaningless configuration that would collapse the subject–value distinction. Together with the $s \neq v$ condition in the statement predicate, these constraints ensure that every DCL instance is a connected, acyclic information graph with clear directionality from subjects to values. The

pandoc/assets/fig_metamodel.png

Figure 3: The three-layer metamodel. The domain concept layer (top) models facts as a typed information graph; the credential schema layer (middle) partitions facts into credentials with subject bindings and trace mappings; the format-specific layer (bottom) assigns concrete formats with capability predicates and governance annotations.

cyclic error predicate enforces acyclicity via transitive closure of the neighbours relation. In terms of the usage modes (??), DCL constraints serve error identification: evaluating non_connected on a partial specification with an orphaned entity returns **NOT_OK**(non_connected(e_1), naming the unreachable pair and indicating where the domain model is incomplete.

In the housing subsidy scenario, the domain concept layer contains:

Element	Type	Description
Applicant	Subject	Person applying for subsidy
has_children	Prop	Family size
num_children	Value	Number of children
owns_property	Prop	Property ownership
property_area	Value	Floor area (m ²)
earns	Prop	Employment income
monthly_income	Value	Monthly net income

Three statements capture the domain facts — one per property edge in the domain graph. Two domain constraints apply: property_area $\geq$ min_area(num_children) links two properties across what will become separate credentials, and monthly_income $\geq$ threshold establishes a privacy-sensitive eligibility check. All claim-layer structural constraints (connectivity, no self-loops) are satisfied.

Table 1: CSL propagation rules that derive type assignments from DCL structure.

Rule	Trigger	Effect	Credential	CredentialSubject	Claim	CredentialValue	Issuer
subject_traces_to_subject	CredEntity traces to DCL Subject	Infer CredentialSubject					
root_is_cred_subj	No Claim targets the CredEntity	Infer CredentialSubject					

Table 2: CSL shadow and error predicates for structural well-formedness.

Predicate	Kind	Condition
credential_statement	Shadow	Claim connects distinct source and target; target typed as CredentialValue
Root_cred_entity	Shadow	CredEntity not targeted by any credential_statement
no_empty_cred	Error	CredentialSubject with no outgoing claim
root_ent_doesnt_have_cred	Error	Root entity without an associated Credential

4.2 Credential Schema Layer

A domain concept graph captures *what* is known about a subject. Credential ecosystems must partition these facts into credentials — each issued by a different authority, each carrying a subset of the domain’s claims. The Credential Schema Layer (CSL) models this partitioning. Each CSL element is traced from a DCL element: the abstract metaclass `CredEntity` carries a mandatory trace reference to exactly one DCL Entity, and each DCL Prop contains exactly one Claim via `Prop::trace`. This pair of cross-layer references ensures that every credential-layer element has a domain-level origin (??). CSL mirrors DCL’s type structure: `CredentialSubject` and `CredentialValue` specialize `CredEntity`, paralleling DCL’s Subject and Value. A Claim connects a source `CredEntity` to a target `CredentialValue`, mirroring DCL’s statement predicate. Each `CredentialSubject` optionally contains a `Credential`, which itself holds exactly one `Formatted_Credential` (??). The complete Refinery class declarations are provided in the supplementary material.

Trace mappings do more than record provenance — combined with Refinery’s propagation rules, they actively derive CSL structure from DCL during model generation. Propagation rules fire when partial information is committed, narrowing the space of valid completions:

The second rule mirrors DCL’s `root_is_subj` at the credential level. The effect is that CSL type assignments are derived from DCL structure rather than independently specified — a credential entity’s type is a consequence of its domain-layer origin. The full propagation rule definitions are provided in the supplementary material.

Beyond type derivation, the metamodel enforces structural well-formedness through error predicates — graph predicates whose satisfaction marks a partial model as inconsistent:

The first two predicates mirror DCL’s statement and root-entity classification at the credential level. The two error predicates catch structural violations that would otherwise propagate silently to downstream format assignment. When either fires, the framework reports the violation — e.g., `NOT_OK(no_empty_cred(cs))` — before format-specific constraints are even evaluated. The full constraint definitions are provided in the supplementary material.

In the housing subsidy scenario, three credentials partition the claim-layer facts:

Credential	CredentialSubject	Claim	CredentialValue	Issuer
FamilyStatusCred	CS_Applicant ₁	has_children ₁	num_children ₁	Civil registry
PropertyCred	CS_Applicant ₂	owns_property	property_area ₁	Land registry
IncomeCred	CS_Applicant ₃	earns ₁	monthly_income ₁	Employer

All three credential subjects trace to the same claim-layer entity: `trace(CS_Applicanti, Applicant)` for $i \in \{1, 2, 3\}$. Each claim traces to its corresponding property, and each credential value traces to its corresponding value. Entity alignment holds pairwise: `aligned(CS_Applicanti, CS_Applicantj)` for all $i \neq j$. The cross-layer domain constraint — minimum floor area as a function of the number of children — now spans two credentials, requiring the verifier to combine claims from `FamilyStatusCred` and `PropertyCred`.

4.3 Format-Specific Layer

The format-specific layer assigns each credential a concrete representation format. The abstract metaclass `Formatted_Credential` has five concrete subclasses (one per credential format in scope: `AnonCreds`, `JSON-LD`, `JWT-VC`, `SD-JWT-VC`, and `mdoc`), and each `Credential` contains exactly one instance via the `format[1]` containment. These formats differ along a design-relevant axis: *selective disclosure* (revealing a subset of claims) versus *predicate proofs* (proving a comparison such as `monthly_income ≥ threshold` without disclosing the value). Only `AnonCreds` supports predicate proofs natively [?]; `SD-JWT-VC` [?] and `mdoc` [?] provide selective disclosure but not predicate proofs, and `JSON-LD` supports selective disclosure via the BBS+ cryptosuite [?]. Formats lacking predicate proofs force issuers to pre-compute boolean claims, restructuring the domain concept layer (??). Unlike DCL and CSL, the format-specific layer does not yet carry intra-layer structural constraints; its role in the metamodel is different. FSL elements participate in cross-layer predicates (??), carry governance annotations as typed containment, and their capability predicates feed the propagation rules that narrow the format design space during generation. The layer is a metamodel component with cross-layer predicate participation and governance semantics, not a lookup table mapping credentials to format labels.

The metamodel encodes format capabilities as six derived predicates over the class hierarchy, each capturing a functional property that at least one governance framework or domain requirement in the running example imposes:

Full definitions are provided in the supplementary material. Propagation rules use these predicates to narrow the format design space during generation: when a capability is required, formats lacking it are eliminated. Requiring predicate proof support on `IncomeCred` eliminates all formats except `AnonCreds`, which in turn does not conform to W3C VCDM 2.0, setting up the governance conflict in ??. Three governance annotation classes (`EidasMandate`, `PrivacyRequirement`, `VcdmConformance`) attach regulatory and standards requirements to individual credentials as typed markers. These annotations are the primary input for error identification

Table 3: FSL capability predicates derived from the format class hierarchy.

Predicate	Capability	#	Constraint	Source	Layers	Housing subsidy instance
supports_predicate_proof	Predicate proof (e.g., $\text{monthly_income} \geq \text{threshold}$ without disclosure)	C1	Entity alignment	Structural	DCL $\leftrightarrow$ CSL	CS_Applicant _{1,2,3} trace to same
supports_selective_disclosure	Selective disclosure of claim subsets					Applicant
conforms_vcdm	W3C VCDM 2.0 data model conformance	C2	Trace consistency	Structural	DCL $\rightarrow$ CSL	Each Claim traces to exactly one Prop
supports_zkp	Zero-knowledge proof support					Each Credential-Subject has ≥ 1 outgoing Claim
supports_offline_verification	Offline verification without network access					property_area $\geq f(\text{num_children})$
supports_multi_credential_proof	Cross-credential predicate evaluation					EU wallet attestations require SD-JWT-VC
		C3	No empty credential	Structural	CSL	Income threshold check $\rightarrow$ predicate proof
		C4	Cross-credential value dep.	Domain rule	DCL horiz.	Format must conform to VCDM data model
		C5	Format mandate	eIDAS ARF	FSL	$C5 \wedge C6 \wedge C7$ unsatisfiable on IncomeCred
		C6	Predicate proof required	GDPR Art. 5(1)(c)	DCL $\leftrightarrow$ FSL	C4 requires cross-credential arithmetic; no deployed format supports it
		C7	VCDM conformance	W3C VCDM 2.0	CSL $\leftrightarrow$ FSL	
		C8	Governance conflict	eIDAS+GDPR+W3C		
		C9	Cross-credential predicate gap	Format limitation	DCL $\leftrightarrow$ FSL	

(flagging format-governance conflicts) and design space exploration (generating only format assignments satisfying all active governance constraints).

In the housing subsidy scenario, format assignments are governance-driven:

Credential	Format	Governance source	Status
FamilyStatusCred	SD-JWT-VC	eIDAS ARF (SHALL)	Valid
PropertyCred	SD-JWT-VC	eIDAS ARF (SHALL)	Valid
IncomeCred	?	eIDAS + GDPR	Conflict

FamilyStatusCred and PropertyCred are straightforward: as EU wallet attestations, the eIDAS Architecture Reference Framework mandates SD-JWT-VC (or mdoc; since both lack predicate proof support, we use SD-JWT-VC as the representative format for this analysis (the two formats differ on other capability predicates such as offline verification) [?]). IncomeCred is the conflict site: the ARF requires SD-JWT-VC, but the income threshold check – verifying $\text{monthly_income} \geq \text{threshold}$ without disclosing the exact value – requires predicate proof capability that SD-JWT-VC does not provide [?]. AnonCreds supports predicate proofs via CL signatures [?] but does not conform to W3C VCDM 2.0 [?].

4.4 Cross-Layer Constraints as Graph Predicates

The following table classifies the constraints exercised in the housing subsidy scenario by source and scope:

Constraints C1–C3 are structural (metamodel-enforced). C4 is a domain rule grounded in government regulation. C5–C7 each originate from a different governance framework. C8 and C9 are cross-layer results: they emerge only when constraints from multiple sources and layers are checked jointly. ?? develops C8 and C9 as the paper’s headline results.

The housing subsidy scenario demonstrates all three usage modes (??) on the cross-layer constraint set. Running consistency checking on the partial specification – FamilyStatusCred and PropertyCred assigned to SD-JWT-VC, IncomeCred unassigned – returns **OK**: structural constraints C1–C3 and domain constraint C4 hold in the partial model. Assigning IncomeCred to SD-JWT-VC and enforcing all governance constraints triggers error identification: the framework returns **NOT_OK(governance_conflict(IncomeCred, income_format))**, naming the credential and format slot where

Cross-layer constraint taxonomy (C1–C9) formatted as captioned figure from the table above.

Figure 4: Cross-layer constraint taxonomy for the housing subsidy scenario. Structural constraints (C1–C3) are metamodel-enforced; domain constraint C4 is grounded in regulation; governance constraints C5–C7 originate from independent frameworks; C8 and C9 are cross-layer results.

C5, C6, and C7 cannot be simultaneously satisfied. Finally, leaving IncomeCred’s format open and running design space exploration with the full constraint set ($C5 \wedge C6 \wedge C7$) returns **UNVIABLE** — no format assignment satisfies all three governance sources simultaneously. Relaxing C6 (dropping the predicate proof requirement), the framework returns two **GENERATED** configurations: both assign SD-JWT-VC to all three credentials, differing in selective disclosure granularity.

The predicates defined in the preceding sections (*non_connected*, *no_empty_cred*, *root_ent_doesnt_have_cred*) operate within a single layer using the Refinery mechanisms introduced in ???. The cross-layer predicates below combine elements from multiple layers and fall into three categories: *propagation rules* narrow the design space during generation by setting inconsistent bindings to **false**; *shadow predicates* derive cross-layer facts without constraining generation; *error predicates* flag violations on committed assignments. This classification determines which usage mode (??) each predicate serves.

A Claim that connects to a CredEntity tracing to the wrong DCL Entity is a silent design error — each layer is well-formed individually, but the cross-layer mapping is broken. Two propagation rules prevent this by negative elimination. The target consistency rule enforces that if $\text{trace}(p, c)$ and $\text{value}(p, e)$ hold (where $\text{trace}(p, c)$ denotes $\text{Prop}::\text{trace}$ and $\text{trace}(t, e)$ below denotes $\text{CredEntity}::\text{trace}$), then any CredEntity t for which $\neg\text{trace}(t, e)$ holds is excluded as c ’s target; the source rule is dual over $\text{property}(e, p)$. In the running example, the Claim tracing to *has_children* can only target a CredEntity tracing to *num_children*, not one tracing to *property_area*. Bindings that are definitely inconsistent with committed traces are set to **false** in Refinery’s four-valued interpretation, removed from the space of valid completions before model generation explores them. The full propagation rule definitions are provided in the supplementary material.

Credentials issued by different authorities for the same person must be recognizable as referring to that person. The shadow predicate *aligned* formalizes this: $\text{aligned}(ce_1, ce_2)$ holds when two distinct CredEntities satisfy $\text{trace}(ce_1, e) \wedge \text{trace}(ce_2, e)$ for some DCL Entity e . In the running example, $\text{aligned}(\text{CS_Applicant}_i, \text{CS_Applicant}_j)$

holds for all $i \neq j$, recording that all three credential subjects represent the same Applicant. A second shadow predicate, *common_parent*, holds when $\text{common_parent}(p_1, p_2)$ is satisfied — i.e., two Props’ traced Claims share a source CredEntity, capturing co-location within a single credential. Both predicates make cross-layer structure visible for conformance checking without constraining generation; *aligned* feeds into *cross_cred_predicate_gap* below, and *common_parent* supports the anti-pattern catalog in ???. Shadow predicates are evaluated in all three usage modes but never trigger errors directly; both predicates inform the anti-pattern analysis.

Some design problems are invisible at any single layer. When a domain constraint spans two credentials — $\text{property_area} \geq f(\text{num_children})$ requires combining claims from FamilyStatusCred and PropertyCred — the design depends on a format capability that may not exist. The shadow predicate *cross_cred_predicate_gap* detects this: $\text{cross_cred_predicate_gap}(c_1, c_2)$ holds when $\text{aligned}(cs_1, cs_2)$ and at least one credential’s format lacks multi-credential proof support. No deployed format supports cross-credential arithmetic (AnonCreds supports multi-credential *presentation* but not cross-credential *computation*), so the predicate fires for every aligned pair, making a structural limitation of the format landscape visible. As a shadow predicate, it records a condition whose severity depends on domain requirements; ?? develops this as the second headline result.

Format capability propagation rules complete the cross-layer mechanism. When a governance annotation or domain requirement demands a capability, propagation rules eliminate incompatible format classes by negative elimination: for a Formatted_Credential f where $\text{supports_predicate_proof}$ must hold, any format class that cannot provide predicate proofs (SD-JWT-VC, JWT-VC, mdoc, JSON-LD) is set to **false**, leaving only AnonCreds. The mechanism is the same as the trace consistency rules (*prop_t*, *prop_s*): Refinery’s four-valued interpretation distinguishes **must** (committed true), **must not** (committed false), **may** (possibly true), and **may not** (possibly false), and propagation rules convert open assignments to committed false when a constraint excludes them. The key distinction is between propagation rules, which guide generation by narrowing the design space, and error predicates such as *governance_conflict*, which check committed assignments and report violations. This distinction enables the metamodel to serve both as a consistency checker for existing designs and as an efficient generative tool for design space exploration.

The contribution of this section is the cross-layer constraint methodology, not the individual layer definitions. Single-layer schemas for credentials and format capability matrices exist in the literature. The novel element is expressing constraints that span metamodel layers as graph predicates evaluable by a partial model solver, detecting conflicts that are invisible to single-layer inspection.

The income threshold check illustrates how format-specific limitations propagate upward through the metamodel. With AnonCreds (CL signatures), the issuer encodes *monthly_income* as an integer attribute; at verification time, the holder proves $\text{monthly_income} \geq \text{threshold}$ via a predicate proof without disclosing the exact value $[?]$ — the domain concept layer structure is unchanged. With SD-JWT-VC, no predicate proof mechanism exists $[?]$. The only workaround is for the issuer to pre-compute boolean claims at issuance: $\text{earns_above_200k} \mapsto \text{true}$, $\text{earns_above_300k} \mapsto \text{true}$,

and so on. This restructures the domain concept layer: the single Prop earns $\rightarrow$ monthly_income (integer) is replaced by multiple Props earns_above_X $\rightarrow$ boolean for each anticipated threshold. The format-specific limitation has forced a change in the domain-level information model — a cross-layer constraint propagation that is visible only when both layers are analyzed together.

5 Evaluation

We evaluate the metamodel and its cross-layer constraint formalization along two complementary axes. A qualitative *elaboration* (??) characterizes metamodel coverage against the W3C VCDM 2.0 specification, constraint expressiveness against EU regulatory sources, two headline results — a governance conflict and a format expressiveness gap — demonstrating cross-layer constraint value, a catalog of structural anti-patterns, and a baseline comparison against single-layer alternatives. A quantitative *scalability measurement* (??) benchmarks three Refinery solver operations across ecosystem sizes up to 30 credentials.

5.1 Elaboration

5.1.1 Metamodel Coverage. We characterize the metamodel’s coverage of W3C VCDM 2.0 concepts [?] as a soundness–completeness pair. For soundness, every metaclass and capability predicate traces to a VCDM concept: DCL metaclasses formalize the claim-level information structure, CSL metaclasses the credential packaging model, and FSL format classes with six capability predicates the format properties relevant to governance constraint evaluation. For completeness, the metamodel deliberately excludes three VCDM concept families outside credential *schema design*: proof mechanisms (cryptographic, orthogonal to structural modeling), verifiable presentations (runtime protocols), and credential status (lifecycle management). These are scope boundaries of the design-time formalization, not limitations of the graph predicate approach. These three families govern how a credential with a given structure is used (signed, presented, revoked), not how that structure is designed. Governance constraints from independent sources (eIDAS, GDPR, W3C) bear on structure and format assignment; formalizing these at design time is what makes cross-source conflicts detectable.

5.1.2 Constraint Expressiveness. We collected eight normative constraints from the eIDAS Architecture Reference Framework (ARF) v2.7.3 [?] governing credential format assignment, and classified each as *fully expressible*, *partially expressible*, or *not expressible*. Three are fully expressible (??), five partially, and none falls entirely outside the metamodel’s capacity. The full constraint analysis is provided in the supplementary material.

The five partially expressible constraints share two root causes. Three constraints (ARF-C2, C3: qualified vs. non-qualified attestation format restrictions; ARF-C6: VCDM completeness as a meta-level property) condition format eligibility on the credential qualification level (PID, QEAA, or non-qualified EAA in eIDAS terminology), but the metamodel does not yet distinguish these attestation subtypes. Two constraints (ARF-C5: per-claim selective disclosability; ARF-C8: salted-hash vs. ZKP-based selective disclosure) require privacy annotations at claim granularity rather than format level. Both gaps are closable by extending the metaclass hierarchy; neither requires changing the constraint formalization approach.

Table 4: Fully expressible eIDAS ARF constraints. Each maps directly to metamodel predicates or architecture.

ID	Constraint	Source	Predicate / Mechanism
ARF-C1	PID must be issued in both ISO 18013-5 and SD-JWT-VC	PID_02	EidasMandate annotation + format class membership
ARF-C4	Proximity presentation requires mdoc format	ARB_02	supports_offline_verification
ARF-C7	Attributes defined encoding-independently, then per-format	ARB_06	propagation rule eliminates SD-JWT-VC DCL $\rightarrow$ CSL $\rightarrow$ FSL layer architecture

A constraint falls outside the metamodel’s capacity only when it governs runtime behavior (revocation timing, holder-binding protocols) with no design-time structural counterpart — none of the eight falls into this category.

Remark. ARF-C7 provides external validation of the metamodel’s layered architecture. The ARF requires that attestation attributes be defined encoding-independently before being specified per-format (ARB_06 [?]). The metamodel was designed from the W3C VCDM structure, not from the ARF; that the EU governance framework independently mandates the same DCL $\rightarrow$ CSL $\rightarrow$ FSL separation confirms the layering reflects a structural property of credential ecosystem design rather than an artifact of the formalization.

5.1.3 Headline Results.

Headline 1: Income governance conflict (vertical). At the credential schema layer, IncomeCred is well-formed: CS_Applicant₃ traces to Applicant, earns₁ traces to the earns property, monthly_income₁ traces to its value. All structural constraints (C1–C3) are satisfied.

At the format-specific layer, three governance sources impose requirements on IncomeCred:

1. **eIDAS ARF** (normative, SHALL): format(IncomeCred) $\in$ {SD-JWT-VC, mdoc} — neither supports predicate proofs [?].
2. **GDPR Art. 5(1)(c)** (authors’ operationalization): the income threshold check requires disclosing only whether monthly_income $\geq$ threshold, not the exact value. We interpret the data minimization principle as requiring that, when a threshold comparison suffices, disclosing the exact value constitutes disproportionate data collection — and that enforcing this at the credential layer requires predicate proof capability [?]. Neither inferential step is self-evident: the first is a reading of data minimization for threshold-check scenarios; the second assumes minimization must be achieved through technical means at the credential layer rather than through organizational or procedural controls.

A directly analogous enforcement action illustrates the operational weight of this principle: the Hungarian data protection authority (NAIH) fined a bank 35M HUF for copying applicants' entire pregnancy booklets when only a 12-week gestation threshold check was required for a subsidized loan. NAIH found the collection grossly disproportionate [?], establishing that collecting exact values when a threshold check suffices violates data minimization — precisely the scenario that predicate proofs address at the credential layer.

3. **W3C VCDM 2.0:** the credential format must conform to the VCDM data model — AnonCreds v1 does not, as its encoding is designed around the CL signature scheme rather than VCDM's data model [?] [?].

No format satisfies all three requirements. SD-JWT-VC satisfies (1) and (3) but not (2). AnonCreds satisfies (2) but not (1) or (3). The configuration is unsatisfiable: constraints C5, C6, and C7 cannot be simultaneously satisfied on IncomeCred.

This contradiction is invisible to single-layer inspection. At the credential schema layer alone, IncomeCred is well-formed. At the format-specific layer under the eIDAS ARF alone, SD-JWT-VC is compliant. At the format-specific layer under GDPR alone, AnonCreds provides the needed capability. Only the joint, cross-governance, cross-layer analysis reveals the conflict.

Remark. An issuer-precomputed boolean claim (`income_above_threshold`: true) can approximate a predicate proof within SD-JWT-VC. However, this workaround requires the issuer to anticipate every verifier threshold at issuance time, produces combinatorial explosion for multi-threshold scenarios, and remains static — a credential issued with threshold X cannot serve a verifier requiring threshold Y without reissuance. As shown in ??, this workaround restructures the domain concept layer — itself a cross-layer propagation that confirms the need for multi-layer analysis.

Headline 2: Cross-credential predicate gap (horizontal). The domain constraint `property_area ≥ min_area(num_children)` (C4) requires combining values from two credentials issued by independent authorities: `property_area` from `PropertyCred` (land registry) and `num_children` from `FamilyStatusCred` (civil registry).

No credential format with a stable specification and production-ready implementations supports cross-credential arithmetic predicates in zero-knowledge. Among the five such formats (SD-JWT-VC, JSON-LD with BBS+, JWT-VC, mdoc, AnonCreds v1), only AnonCreds v1 offers any predicate support — single-credential range proofs (`attr ≥ const`) under CL signatures — but even it cannot combine values across credentials issued by independent authorities. ZK-SNARK-based research prototypes such as `zk-creds` [?] demonstrate cross-credential predicates but lack EU wallet ecosystem adoption and W3C VCDM conformance. AnonCreds v2 plans BBS+ signature support with range proof capabilities, but no stable specification is available for independent verification.

To verify the floor area constraint, the verifier must see both raw values from two separate credentials, defeating the privacy properties that ZKP-capable formats promise. The metamodel captures this: a domain-concept-layer constraint (C4) that spans credentials cannot be enforced privacy-preservingly at the format-specific layer

because no deployed format supports cross-credential predicate proofs (C9).

This gap is again invisible to single-layer inspection: the domain concept layer constraint is well-defined, both credentials are well-formed at the credential schema layer, and each credential's format is individually valid at the format-specific layer. Only the cross-layer analysis — checking whether the DCL constraint can be enforced given the FSL format capabilities — reveals the expressiveness gap.

Complementarity. The two results are orthogonal. Headline 1 identifies a *vertical* governance conflict: contradictory requirements on a single credential's format from different regulatory sources. Headline 2 identifies a *horizontal* expressiveness gap: an ecosystem-level constraint spanning credentials that exceeds any single format's capabilities. Together, they demonstrate that multi-layer analysis detects both governance conflicts and format expressiveness gaps invisible to single-layer inspection.

5.1.4 Anti-Pattern Detection. ?? catalogues five structural anti-patterns formalized as graph predicates over the partial model.

Anti-pattern	Description	Graph predicate	Layer(s)	Kind
Disconnected domain graph	Entity pair not transitively reachable	<code>non_connected</code>	DCL	error
Empty credential	Credential subject with no outgoing Claim	<code>no_empty_cred</code>	CSL	error
Orphaned root entity	Root Credential without associated Credential	<code>root_ent_doesnt_have_cred</code>	CSL	error
Trace misalignment	Claim target traces to wrong DCL Entity	<code>prop_t / prop_s</code>	DCL ↔ CSL	propagation
Cross-credential predicate gap	Aligned credential pair lacks cross-credential proof support	<code>cross_cred_predicate_gap</code>	DCL ↔ FSL	shadow

The first three anti-patterns are detectable by single-layer inspection: `non_connected` operates within the DCL, `no_empty_cred` and `root_ent_doesnt_have_cred` within the CSL. Trace misalignment requires cross-layer analysis. Each layer is individually well-formed, but the propagation rules `prop_t` and `prop_s` eliminate any binding where a Claim's target Credential traces to a different DCL Entity than the Prop it was derived from. The cross-credential predicate gap is invisible at any single layer: the domain constraint is well-defined at the DCL, both credentials are structurally valid at the CSL, and each format is individually compliant at the FSL. Only the cross-layer shadow predicate, which checks aligned credential subjects against their formats' `supports_multi_credential_proof`

capability, surfaces the gap. This graduated visibility, from intra-layer errors through cross-layer trace inconsistencies to ecosystem-level capability gaps, is the central argument for multi-layer formalization over single-layer alternatives. The catalog is extensible: adding an anti-pattern requires a new graph predicate over the existing metamodel, not structural changes to layers or trace links.

5.1.5 Baseline Comparison. The multi-layer advantage for cross-layer detection is partly definitional. The comparison’s purpose is to identify which issue classes fall into this detection gap and to confirm that current practice cannot cover them. We compare analytically against three baselines of increasing formality; no existing tool implements cross-layer credential ecosystem checking, so the comparison is structural rather than empirical. Manual expert review, the current industry practice for credential ecosystem design, can identify single-credential format conflicts but lacks systematic coverage of cross-credential dependencies and provides no guarantee that all governance sources have been jointly checked. Formalization adds systematicity: the model checks all constraint combinations exhaustively (the G0–G7 sensitivity experiment confirms only the full conjunction produces unsatisfiability), scales beyond what manual analysis can track, and yields a reproducible artifact. Single-layer metamodeling (e.g., a UML class diagram with OCL constraints per layer) detects intra-layer structural violations such as empty credentials or disconnected domain graphs, but cannot express the cross-layer trace predicates (`prop_t`, `prop_s`) or the cross-layer capability checks (`cross_cred_predicate_gap`) that link domain constraints to format capabilities. Only the integrated multi-layer formalization with cross-layer graph predicates detects all five anti-pattern categories, including the two headline results (??) that are invisible to any single-layer approach. No alternative multi-layer credential ecosystem formalization exists in the literature; the three baselines represent the state of practice, not a weakened comparison set.

5.2 Scalability Measurement

We evaluate the scalability of the Refinery-based formalization across three solver operations of increasing cost. *Consistency checking* (check in Refinery) verifies that the partial model specification has no internal contradictions. *Concretizability checking* (check -k) goes further: it determines whether a concrete model satisfying all constraints, including error predicates, exists — this is the operation that detects governance conflicts such as the income format unsatisfiability in ??. *Model generation* (generate) produces a fully resolved model instance, enumerating valid credential ecosystem designs for design space exploration. We measure all three to answer two research questions. **RQ1:** How does conflict detection (concretizability checking) scale with model size? **RQ2:** How does design space exploration (model generation) scale with model size? Consistency checking serves as a baseline: it should scale well but cannot detect cross-layer conflicts, because the partial model is internally consistent even when no valid concretization exists.

We construct synthetic instances from $N=1$ to $N=30$ credentials. Each credential contributes one Prop-Value pair at the DCL, one CredentialSubject-Claim-CredentialValue-Credential group at the CSL, and one Formatted_Credential node at the FSL, yielding a total of 11 to 272 graph nodes (?). All credential

Table 5: Scalability measurements for two Refinery operations, baseline-corrected (3.9s Docker+JVM overhead subtracted). *Concretizability* (check -k): determines whether a concrete model satisfying all constraints exists. *Generation* (generate): produces a concrete model instance. Seconds, mean $\pm \sigma$ over 10 runs.

N	$ V $	SAT (s)	UNSAT (s)	Generation (s)
1	11	<0.1	<0.1	0.55 ± 0.11
3	29	<0.1	0.18 ± 0.07	0.56 ± 0.05
5	47	0.13 ± 0.10	0.11 ± 0.05	0.69 ± 0.08
10	92	0.14 ± 0.05	0.22 ± 0.07	0.77 ± 0.06
15	137	0.42 ± 0.12	0.40 ± 0.08	0.94 ± 0.05
20	182	0.51 ± 0.06	0.58 ± 0.09	1.23 ± 0.07
30	272	1.09 ± 0.06	1.19 ± 0.06	1.85 ± 0.07

subjects trace to a shared Subject; the last credential’s format class is left unresolved for the solver. Each scale point has two variants: a satisfiable (SAT) variant without governance conflicts, and an unsatisfiable (UNSAT) variant that imports the Headline 1 governance conflict predicate, requiring the solver to detect that no format assignment satisfies all three governance frameworks simultaneously. As a secondary *constraint sensitivity* experiment, we fix $N=3$ and vary governance framework combinations over the power-set $\mathcal{P}(\{\text{eIDAS}, \text{Privacy}, \text{VCDM}\})$, yielding eight configurations (G0–G7). Instance definitions and Refinery encodings are provided as supplementary material.

All instances are evaluated using the Refinery CLI,² where each invocation starts a fresh JVM inside a Docker container. We use Hyperfine as the benchmarking harness with 10 measured runs and 1 warmup run per configuration, on an AMD Ryzen 9 7950X3D (16 cores), 96 GB RAM, Windows 11. Cold JVM startup inside the Docker container adds a constant overhead of ≈ 3.9 s per invocation (measured via a no-op baseline instance); reported times subtract this overhead to isolate solver computation. At small scales ($N \leq 5$), solver time is below the measurement noise floor (≈ 0.1 s). The measurement script, generated instances, and the metamodel source are provided as supplementary material for independent reproduction.

RQ1 (conflict detection scaling): Concretizability checking scales sublinearly with model size, rising from below the measurement noise floor at $N=1$ to 1.09 s at $N=30$ (SAT). SAT and UNSAT instances exhibit comparable timing at each scale point, with differences within measurement noise (σ); at the largest scale ($N=30$), the UNSAT overhead is 0.10 s, confirming that unsatisfiability detection does not incur significant additional cost. At $N=30$ (272 graph nodes), conflict detection completes in under 1.2 s — well within interactive use for credential ecosystem sizes an order of magnitude beyond current EU wallet specifications. **RQ2** (design space exploration scaling): Model generation follows the same growth trend, rising from 0.55 s to 1.85 s. The higher baseline reflects the cost of producing a fully resolved model instance rather than merely determining satisfiability.

²Container image ghcr.io/graphs4value/refinery-cli, pulled via Docker.

Scalability plot: wall-clock time (s) vs. model size (N credentials). Lines: consistency check, concretizability-SAT, concretizability-UNSAT, model generation-SAT.

Figure 5: Runtime of three Refinery solver operations across ecosystem sizes from $N=1$ to $N=30$ credentials. Consistency checking serves as baseline; concretizability checking detects governance conflicts; model generation produces valid configurations.

Table 6: Constraint sensitivity at $N=3$: governance framework power-set. Result column reports concretizability (check -k). All configurations complete in under 0.1s of solver time (below the measurement noise floor at this scale).

Config	eIDAS	Privacy	VCDM	Result
G0				SAT
G1	x			SAT
G2		x		SAT
G3			x	SAT
G4	x	x		SAT
G5	x		x	SAT
G6		x	x	SAT
G7	x	x	x	UNSAT

The constraint sensitivity experiment confirms that only G7, the conjunction of all three governance frameworks, yields unsatisfiability; all seven proper subsets are satisfiable (??). This validates the Headline 1 finding (??): the income governance conflict requires the simultaneous imposition of eIDAS ARF format mandates [?], GDPR data minimization requirements [?], and W3C VCDM conformance [?]. No proper subset of these three sources produces a conflict. The measurements cover N up to 30 credentials (272 graph nodes); for reference, the EU Digital Identity Wallet Architecture Reference Framework defines fewer than 10 attestation types in its current version [?], though future ecosystem growth may increase this number. The contribution is the metamodel and its cross-layer constraint formalization; Refinery serves as the validation vehicle, and absolute performance numbers are tool-specific.

5.3 Threats to Validity

Construct validity. The metamodel formalizes credential *schema design*, not the full credential lifecycle. Three VCDM 2.0 concept families fall outside this scope: proof mechanisms (cryptographic layer, orthogonal to structural modeling), verifiable presentations (runtime holder-verifier protocols), and credential status (issuance and revocation lifecycle). These boundaries constrain the class of expressible governance requirements to those that condition format assignment on structural or capability properties of credentials.

Within this scope, five of eight eIDAS ARF constraints are classified as partially expressible (??). The two root causes are a missing attestation-type subclass hierarchy (three constraints condition format eligibility on credential qualification level) and per-claim rather than per-format privacy annotation (two constraints require claim-level selective disclosure control). Both gaps are closable by metaclass extension without changing the constraint formalization approach, but the partially-expressible classification itself rests on our judgment of what constitutes a structural versus a runtime property. Additionally, the DCL’s structural inference rule classifies any Entity with no incoming value reference as a Subject; if a modeler omits a property edge by mistake, the entity may be silently promoted to Subject rather than flagged as incomplete.

Internal validity. The running example (housing subsidy) was selected for structural completeness: it exhibits both a vertical governance conflict (Headline 1) and a horizontal expressiveness gap (Headline 2) within a single, independently documented domain. A randomly sampled credential ecosystem might expose neither or might expose interaction patterns not covered by the current anti-pattern catalog (??). The eight eIDAS ARF constraints were extracted from a specific version (ARF v2.7.3 [?]); the constraint landscape may shift as the regulatory framework evolves. Format capability predicates depend on the characterization being both complete and current: an emerging format with novel capability combinations (e.g., BBS+ credentials with partial predicate support) would require extending the format class hierarchy and the derived propagation rules. The scalability instances grow by adding credentials with uniform structure (one property per credential, shared subject), which reflects the common multi-issuer credential ecosystem pattern but does not exercise deeper claim hierarchies or multi-subject credentials that may arise in domains such as healthcare or supply chain management.

External validity. The evaluation operates within a single governance context: EU regulations (eIDAS 2.0 [?], GDPR [?]) applied to Hungarian administrative procedures. The GovernanceAnnotation mechanism is designed to be governance-agnostic: new regulatory sources require adding annotation subclasses and error predicates,

not restructuring the three-layer architecture. However, the evaluation demonstrates this only for the EU context. Governance traditions that impose constraints not reducible to format-capability requirements, such as organizational trust hierarchies or issuer accreditation rules, would require extending the metamodel beyond annotation markers. The single-domain scenario exercises a three-credential, single-subject ecosystem; domains with richer claim structures or multi-subject credentials may stress different metamodel elements. The intended user is a credential ecosystem designer who specifies the domain graph, credential decomposition, and governance requirements in a graph-based formalism; whether a domain expert without metamodeling experience could use the approach effectively remains untested.

Conclusion validity. The scalability instances are synthetic: each adds a credential with one property to the preceding configuration. This linear, homogeneous growth pattern is representative of the common case where independent issuers each contribute one credential to a shared ecosystem, but it does not capture ecosystems where a few credentials carry many claims while others are minimal. Concretizability checking scales sublinearly over the measured range, though growth is uneven — a step around $N=15$ suggests phase transition behavior in the solver’s search strategy. Model generation also scales sublinearly ($3.4\times$ increase across a $25\times$ growth in model size), though superlinear growth may emerge at larger scales as the combinatorial format assignment search dominates. The constraint sensitivity analysis partially compensates for the homogeneous scaling by varying governance complexity at fixed model size, but the power-set covers three governance frameworks only. Refinery-specific performance results establish that the formalization is computationally feasible at practical ecosystem scales; they do not generalize to other partial modeling tools. The contribution is the metamodel and its cross-layer constraints, with Refinery as the validation vehicle. Model definitions, constraint encodings, and all measurement artifacts are provided as supplementary material for independent reproduction.

6 Related Work

6.1 Credential Ecosystem Design and Formalization

Verifiable credential schema design is currently guided by specifications that define credential structure at individual abstraction layers: the W3C Verifiable Credentials Data Model 2.0 [?] and EU Architecture Reference Framework [?] prescribe issuance and presentation flows, while AnonCreds [?] and ISO mDL [?] define format-specific encoding and proof mechanisms. None of these specifications provides a formal mechanism for checking constraints that span multiple layers. Where formal methods have been applied, they target individual layers: Braun and Käfer [?] define RDF-based semantics for selective disclosure and zero-knowledge proofs on verifiable credentials, Yamamoto et al. [?] formalize selective disclosure specifically for linked-data credentials, and Braun et al. [?] verify SSI protocol security properties using ProVerif. Each formalization targets protocol-level security or single-format semantics — none operates across the boundary between domain-level claim semantics and format-specific representation capabilities.

Complementary conceptual models address individual concerns in credential system design. Tith and Colin [?] propose a trust policy meta-model that captures how identity systems establish and evaluate trust across organizational boundaries. Turkanović et al. [?] formalize delegated authority enforcement, modeling how credential issuance rights propagate through delegation chains. The Trust over IP stack [?] organizes the credential ecosystem into informal governance layers, and Naghmouchi and Laurent [?] systematize privacy-by-design principles for SSI. Each of these works addresses a single concern — trust policy, delegation, governance layering, or privacy — but none formalizes cross-layer constraints spanning domain semantics, credential structure, and format-specific representation simultaneously.

6.2 Model-Driven Engineering for Security and SSI

Model-driven security engineering has established metamodel-based approaches to security properties of software architectures. UMLsec [?] annotates UML models with confidentiality and authentication constraints, enabling formal verification of security properties during design. SecureUML [?] integrates role-based access control specifications into class models, generating enforcement infrastructure from the model. Neither targets credential schema design — both operate on software architecture elements rather than the domain-specific structure of verifiable credentials.

Within SSI specifically, four model-driven approaches address different facets of the design problem. Cippitelli et al. [?] model SSI interactions as BPMN choreographies and generate executable code from the choreography models — their concern is the protocol execution flow between participants, not the structure of credentials exchanged. Ding and Sato [?] apply model-driven security analysis to SSI architectural patterns, systematically identifying threats in system configurations rather than checking credential design consistency. Pattiyanon et al. [?] define domain-specific modeling languages and knowledge graphs that surface common SSI implementation weaknesses, while Barclay et al. [?] model SSI governance requirements using iStar goal models, operating at the requirements level without formalizing credential structure. In adjacent work, King et al. [?] automate multi-level governance compliance checking using institutional norms — a structurally related problem, though their formal objects (normative rules governing agent actions) differ from the structural constraints over model elements addressed here.

Model-driven engineering has thus been applied to SSI for choreography, security analysis, weakness detection, and governance requirements. None of these works defines a multi-layer metamodel or formalizes cross-layer constraints connecting domain-level semantics through credential structure to format-specific representation.

6.3 Multi-Level Modeling and Graph-Based Design Space Exploration

Multi-level metamodeling, as established by Atkinson and Kühne [?], eliminates accidental complexity in deep classification scenarios by allowing model elements to span more than two metalevels; de Lara and Guerra [?] implement these principles with deep

instantiation in MetaDepth. Diskin et al. [?] formalize consistency checking across heterogeneous model views — a problem this work extends by adding independently governed constraint sources as a consistency dimension.

The present metamodel relies on the Refinery partial graph modeling framework [?] as its solver infrastructure. Refinery generates diverse model instances that satisfy structural and relational constraints expressed as graph predicates, and its four-valued partial model semantics enable reasoning over designs that are still incomplete — a property essential for credential ecosystems where not all schema elements are known at design time. Farkas et al. [?] applied Refinery to credential schema validation with a single-layer prototype; the present work extends this to a three-layer metamodel with formalized cross-layer constraints.

Alloy [?] performs bounded model finding over relational specifications but operates on fully concrete instances within a fixed scope and cannot represent open design decisions. It relies on SAT-based enumeration rather than incremental constraint propagation, re-encoding the entire problem for each candidate instance. OCL constraints on UML metamodels similarly assume fully instantiated models, with no representation of open decisions. Refinery’s partial model semantics distinguish definite violations from open decisions through four-valued evaluation and graph-query-based constraint propagation, enabling reasoning over specifications where governance constraints from different sources arrive incrementally.

Unlike standard multi-level modeling applications where layers represent successive instantiation and constraints take the form of potency annotations, the three layers in the present metamodel — domain concepts, credential structure, and format-specific representation — represent independently governed concern spaces. They are connected by coverage and capability constraints formalized as graph predicates over partial models (??), not by instantiation relationships. The contribution is therefore the cross-layer constraint methodology under multi-source governance, not the layering technique itself: constraints originate from normative sources — W3C specifications, EU regulations, national privacy legislation — that impose requirements independently and may be formally irreconcilable (??).

No prior work combines multi-layer metamodeling with formalized cross-layer constraints for verifiable credential ecosystem design under multi-source governance.

7 Conclusion

This paper showed that a multi-layer formal model reveals cross-layer design conflicts in credential ecosystems that are invisible to single-layer inspection. The three-layer metamodel (DCL, CSL, FSL), grounded in VCDM 2.0 and formalized as graph predicates in Refinery (??), makes cross-layer constraints from heterogeneous governance frameworks jointly evaluable — supporting consistency checking, error identification, and design space exploration (??). A three-axis validation confirmed coverage against the W3C specification, expressiveness against regulatory and standards sources, and error visibility against known design anti-patterns (??). Notably, the eIDAS Architecture Reference Framework independently mandates encoding-independent attribute definition before per-format specification (??), providing external validation that the layered

architecture reflects a structural property of credential ecosystem design.

The coverage characterization shows that the metamodel captures the core structural concepts of VCDM 2.0 relevant to credential schema design, with excluded concepts (proof mechanisms, revocation, presentation exchange) motivated by the design-time scope (??). The income credential governance conflict demonstrates that cross-layer constraints from eIDAS, GDPR, and VCDM 2.0 conformance requirements are jointly unsatisfiable — no single credential format meets all three governance mandates simultaneously (??). The cross-credential predicate gap shows that a domain-level constraint spanning two independent credentials exceeds the capabilities of all deployed credential formats, forcing raw value disclosure (??). The formalization additionally captures five structural anti-patterns as graph predicates, establishing graduated error visibility from intra-layer well-formedness through cross-layer trace consistency to ecosystem-level capability gaps (??).

The format-specific layer (??) is less mature than the domain concept and credential schema layers, with fewer constraints formalized and fewer credential formats modeled. The constraint catalog (??) is representative rather than exhaustive, drawn from a subset of EU regulatory sources (eIDAS Architecture and Reference Framework, GDPR) rather than a comprehensive governance survey. The approach has not been evaluated empirically with credential ecosystem designers in practice. The formalization relies on a single tool, Refinery (??); portability to other partial-model solvers or constraint-based toolchains has not been assessed. Finally, the metamodel addresses design-time concerns only — runtime credential exchange, revocation processing, and deployment validation remain out of scope.

Four directions follow from the limitations above. First, the format-specific layer should be extended with additional formats and richer capability constraints to match the maturity of the upper layers. Second, the constraint catalog should be broadened beyond EU regulatory sources to include national eIDAS implementations and sector-specific governance. Third, an empirical case study with credential ecosystem designers would evaluate whether the formalization improves design decision-making in practice. Fourth, portability to other constraint-based toolchains (Alloy, USE/OCL) merits consideration to reduce the current single-tool dependency. The income governance conflict and the cross-credential predicate gap (??) show that cross-layer design errors are structural consequences of multi-source governance, not edge cases; formal multi-layer modeling provides a systematic means of identifying them before deployment.